Team plan from now to hack day

Shane, Atif, Edwin, Ruthran and Vincent · Thursday 5 November 2026 · replaces the task order in the playbook and in docs/backend-plan.md. The live task table is AGENTS.md in the repo. Hack days are Wednesday 11 to Friday 13 November.

Where we are

- **Built and on GitHub:** the interface, the FastAPI server, the engine, the offline reasoner, 25 backend tests, 3 chat tests, the interface gate, a real-data check script and a fake SEG-Y survey for laptops.
- **Not done yet:** nothing has run on the real HPC data, and Gemma has not been tested. Each open bug below is a task.
- **Targets:** the first real-data answer on Friday 6 Nov. Version 1 with Gemma on real data, checked by Ruthran, by Saturday 7 Nov. Feature freeze on Tuesday 10 Nov at 8 pm.
- **Every coding agent reads AGENTS.md:** Codex and Cursor read it on their own, and Claude Code reads it through CLAUDE.md. So all three coding machines follow the same rules.

The team

Person	Role	Machine and agent	Owns
Shane spicydood	Product, interface, integration, HPC runs, final merges, drives the demo	Mac, Claude Code	frontend/, docs/, README, AGENTS.md, interface checks
Atif AtifAsrul	Backend and AI agent	Windows, Codex	backend/, eval harness, backend tests
Edwin edwindows11	Data and evidence engine	Laptop, any agent that reads AGENTS.md	engine/, config.py, the real-data check, the fake SEG-Y survey, engine tests
Ruthran ruthran22003223	Geoscience lead and data QC. Final say on every geological claim. Works at DUG.	HPC desktop (DUG Insight), GitHub website	Knowledge cards, reservoir window, well results, reference answers, scoring
Vincent vinsoontian	Presenter, slides, business case, question bank	Slides tool, GitHub website	The deck, the pitch script, the business-impact model, question drafts

What the judges score

Criterion	Weight	Carried by
Business impact	20 %	Vincent, Ruthran
Technical design · Technical execution	15 % each	Atif, Edwin, Shane
User experience · Solves the user's challenge	15 % each	Shane · Ruthran, Atif
Presentation · Innovation	10 % each	Vincent, Shane · everyone

How the system works

- **Edwin's engine** reads the real SEG-Y and well files on the HPC once, and writes a store of measurements outside the repo.
- **Ruthran** sets what it measures: the reservoir window, the well results and the sand threshold. He checks the result against DUG Insight.
- **Atif's server** answers each question. Its tools pull evidence items E1, E2 and so on from the store.
- **Gemma 4** writes the explanation from that evidence only. It never reads data files, and it is reachable only from the HPC.
- **Shane's interface** streams the answer and links every evidence tag to the map and the section.
- **Vincent** turns it into a 10-minute story with the business case. **Ruthran** answers the geology questions in Q&A.

Hand-offs

Pair	What passes between them	Agree before changing
Ruthran and Edwin	What to measure, the window, thresholds, well results, well positions	Lead rules or scores
Edwin and Atif	The store layout: meta.json, wells.json, leads.json, maps, cubes	Any store field (gate 3)
Atif and Shane	API routes and chat events: status, tool, evidence, token, reset, done	Any field or event (gate 3)
Ruthran and Vincent	The question bank: Vincent drafts, Ruthran answers and scores	Nothing
Vincent and Shane	The demo script, the screenshots, the timing	Nothing

One 15-minute stand-up a day, online is fine: what I did, what I'm doing next, what blocks me.

The three gates

Every coding task follows this path. Shane runs gates 1 and 2 with Claude Code. Nothing reaches main without both.

Step	Who	What happens
1. Plan	Task owner	Five lines or fewer: task ID, files, approach, the test you will add, estimate.
Gate 1	Shane + Claude	Claude checks the plan against AGENTS.md and the code. Go, or change it.
2. Build	Task owner	On a branch named owner/task-id-name. Add a test that fails before the fix and passes after.
3. Laptop tests	Task owner	Run the laptop tests for your area. Paste the pass counts into the pull request.
Gate 2	Shane + Claude	Claude reviews the pull request, and the owner fixes what it finds. Then Shane merges. Atif or Edwin reviews Shane's own pull requests.
4. HPC	Shane	git pull on the HPC, restart, run the HPC tests.
Gate 3	Both sides	Store fields, API fields and chat events are shared. Agree on them before writing code.

```
Gate 1 - E2 - plan
Files: engine/io.py, engine/precompute.py
Approach: read only the samples inside the window, one volume at a time
Test: tests/test_engine.py - windowed read equals the full read inside the window
Estimate: 2.5 h

Gate 2 - E2 - PR #<number>
Tests: unittest 27 pass, eval 10/10, fake SEG-Y loop passed, browser checklist done
```

Schedule

Estimates are working hours. Waiting on someone else or for an HPC slot is not counted.

Day	Building: Shane, Atif, Edwin	Geology and pitch: Ruthran, Vincent	Checkpoint
Thu 5 Nov evening	Shane S0 HPC setup Atif A0 setup, A2 winding fix Edwin E0 setup	Vincent V4 email the organisers Ruthran starts R1	Everyone set up, tests pass
Fri 6 Nov	Shane + Edwin S1 first real run Atif A1 opportunity Edwin E2 windowed read	Ruthran R1 window and results Ruthran + Edwin E1 well positions Vincent V1 deck skeleton	First real-data answer on the HPC
Sat 7 Nov	Atif + Shane A4 Gemma on the HPC Edwin E3 safe rebuild, E4 provenance	Ruthran R2 QC against DUG Insight Vincent V1 script, V2 business case	Version 1: Gemma on real data, checked by Ruthran
Sun 8 Nov	Atif + Shane A5 follow-ups Edwin E5 threshold, E6 engine tests	Vincent + Ruthran V3 + R4 30 questions and reference answers Vincent V2 finished	Follow-ups work, question bank ready
Mon 9 Nov	Atif A6 blind well test Shane S4 Gemma eval on the HPC Edwin fixes from R2	Ruthran R4 scores blind, R3 cards and rules Vincent slides with screenshots	Eval score recorded, blind test ready
Tue 10 Nov	Shane S3 freeze at 8 pm, backup video Atif and Edwin: fixes from the eval only	Vincent V5 rehearsal 1 Ruthran R5 checks slides, Q&A prep	Frozen, backup video recorded
Wed 11 to Fri 13 Nov	Day 1: venue and RustDesk test, show mentors Day 2: fix what mentors flag, re-record the video	Day 3: two rehearsals, V6 submit slides and repo link, present for 10 min, then 5 min of Q&A	Present on Fri 13

Hours per person

Person	Total	Made of
Shane	8 to 10 h	S0, S1, his halves of A4 and A5, S4, S3
Atif	12 to 16 h	A0, A2, A1, A4, A5, A6, his half of E4
Edwin	10 to 12 h	E0, his half of S1, E1 to E6
Ruthran	10 to 11 h	R1 to R5, his halves of E1 and E5, Q&A drills
Vincent	about 10 h	V1 to V6

If time runs short

- **Must have:** S0, S1, E1, E2, R1, R2, A1, A4, E3 and V1. Together they give a real-data answer that Ruthran will defend, and a deck.
- **Should have:** A5, E5, R4, V2 and V3. Judges ask follow-ups, and business impact is 20 % of the score.
- **Nice to have:** A6, the blind well test. Show it only if the hidden result is provably excluded.

Tasks: Shane

S0 HPC setup

Shane · 30 min

Do	Follow docs/team-setup.md, HPC part 1: clone into /data/epic20/APGCE/apgce_team_06, environment in team storage, pip install.
Done when	python scripts/check_real_data.py starts without import errors.

S1 First real run, with Edwin

Shane + Edwin · 1.5 to 3 h

Do	Run check_real_data.py and screenshot the output to Claude. Fix any FAIL with Edwin. Run precompute with the provisional window, then render_instrument, then the server. Ask the headline question, "Rank the leads" and one well question.
Watch	If precompute is killed or very slow, stop and switch to E2, the windowed read.
Done when	/api/health shows mock false, llm_mode gemma, llm_reachable true. Answer footers say "Gemma 4".

S4 Gemma eval on the HPC

Shane + Ruthran · 3 h

Do	Run python eval/run_eval.py --questions eval/questions.csv --tag gemma-<date> on the HPC. Ruthran scores the sheet blind. The three lowest answers become tasks for Atif or Edwin.
Rule	The sheet holds answers about real data, so it stays on the HPC. Only the average score leaves, for the slides.

S3 Freeze and backup

Shane · 3 h

Do	Freeze at 8 pm on Tuesday 10 Nov. Run MOCK=1 python frontend/build_standalone.py for the offline demo file. Record a backup screen video of the full demo on the HPC. Drive the demo in every rehearsal.

Tasks: Atif

A0 Pull main and align Codex

Atif · 20 min

Do	git checkout main, git pull. Codex reads AGENTS.md on its own. Run laptop tests 1 to 5.

A2 "Winding" only when the channel actually winds

Atif · 30 min

Problem	backend/agent.py, around line 281, always says "elongate and winding". A lead on the synthetic survey has sinuosity 1.0, which means a straight body.
Do	Say "winding" only when sinuosity is at least Params.sinuosity_channel (1.3). Otherwise say "elongate", or "compact" if elongation is low too.
Test to add	Sinuosity 1.0 never gives "winding". Sinuosity 1.5 does.

A1 "Opportunity" means an undrilled lead

Atif · 2 h

Problem	With a drilled lead selected, "What is the geological opportunity, and why?" calls that drilled lead the opportunity.
Do	For opportunity or drill-next questions with a drilled lead selected, answer about the best undrilled lead, and say the selected lead is a drilled calibration lead. "Tell me about L1" still uses L1. Check the Gemma planner prompt in backend/prompts.py for the same mistake.
Test to add	A drilled lead is selected and the headline question is asked. The answer names the undrilled lead, and "Tell me about L1" still uses L1.

A4 Gemma on the HPC

Atif + Shane · 2 to 3 h

Do	In a slot with Shane, set LLM=gemma on the real store. Run the 10 template questions, with real well names filled in on the HPC, plus the browser checklist. Count the "Gemma couldn't plan" and "stopped partway" footers, and time each answer. Tune backend/prompts.py and LLM_TIMEOUT_S.
Done when	8 of 10 answers come from Gemma with no planner fallback, each takes under 30 s, and no answer has numbers missing from the evidence.

A5 Follow-up questions

Atif, chat side by Shane · 3 to 4 h

Gate 3	Proposal: an optional history field on /api/chat holding the last two turns as {question, lead}. The chat sends it, and the server stores nothing.
Do	Questions with no lead named that read as follow-ups ("why", "what about", "and the") reuse the previous lead and topic. "The seal" maps to that lead's risk tools. Selecting a new lead resets the context.
Tests to add	"Why?" after an L3 answer fetches L3 evidence. "What about the seal?" fetches L3 risks. After L1 is selected, "Why?" uses L1.

A6 Blind well test, for real

Atif · 4 to 6 h

Problem	hide_wells is accepted but ignored. The interface only hides the well on screen.
Do	Build a per-request view of the store without the hidden well's result, log summary or result-based labels, across every tool, the planner's well list, the calibration averages and the cache key. Seismic values at the well stay. Then set hide_wells to true in /api/health.
Test to add	No tool output and no Gemma prompt contains the hidden well's result or log numbers. Changing only its result changes nothing before the reveal.

Tasks: Edwin

E0 Set up and learn the fake survey

Edwin · 30 min

Do	Clone, set up the venv, pip install -r requirements-dev.txt. Run laptop tests 1 and 4, and the fake SEG-Y loop in AGENTS.md (Edwin's section). That loop is your daily test bench: it lays out SEG-Y, LAS and the well list exactly like the HPC.

E1 Wells where DUG Insight puts them

Edwin + Ruthran · 1 to 2 h

Problem	Wells are placed from the well list X and Y, or from the lower third of the deviation path. A different coordinate system or an unrecognised column puts wells in the wrong place.
Do	Compare check_real_data.py's well table and the app's map with Ruthran's positions in DUG Insight. Fix column detection or placement in engine/wells.py.
Done when	Every well is within one bin (25 m) of its DUG Insight position, or the reason is written down.

E2 Read only the window

Edwin · 2 to 3 h

Problem	engine/io.py read_segy loads every trace's full sample range for all 9 volumes. check_real_data.py prints each volume's size. If the total approaches the login node's 128 GB, precompute gets killed.
Do	Read only the samples the engine uses, the window plus whatever padding the section cubes need, one volume at a time, freeing each before the next. Check in precompute.py which time range the cubes keep before choosing the padding.
Test to add	On the fake survey, the windowed read equals the full read inside the window, and the store is unchanged.

E3 Rebuild the store without deleting the working one

Edwin · 1.5 h

Problem	engine/precompute.py deletes the output folder before it builds. A crash during the demo leaves the app with no data.
Do	Build into copilot_store.new. Swap it in only on success, and keep the old store when anything fails.
Test to add	A build that fails halfway leaves the old store intact with no .new folder. A good build leaves no .old folder.

E4 Say which build the app is showing

Edwin + Atif · 1 h

Gate 3	Agree the field names with Atif first.
Do	Write the window, the thresholds, the build time and the git commit into meta.json. Show them in /api/meta, so a screenshot proves which build a demo used.

E5 One calibrated sand threshold

Edwin + Ruthran · 1.5 h

Problem	precompute passes None to build_maps, so --sand-threshold never reaches the top-sand surface. The leads and the surface can use different thresholds.
Do	Pass the chosen threshold through. With Ruthran, pick it from the sand probability at discovery wells versus dry wells (meta.json, calibration).
Test to add	With --sand-threshold 0.6, meta.json, the leads and the top-sand surface all use 0.6.

E6 Engine tests

Edwin · 1.5 h

Do	Write tests/test_engine.py on the fake SEG-Y survey: header bytes and grid, windowed read, well placement, safe rebuild. These guard every later engine change.

Tasks: Ruthran

R1 Reservoir window and well results

Ruthran · 1 h

Do	In DUG Insight, open an inline through the channel at a discovery well. Read the two-way time at the top and the base of the reservoir sand, and send Shane a start and an end in ms with about 20 ms of margin. Then list each of the 13 wells as discovery, dry or unknown.
Where it goes	Shane or Edwin puts the results in data/well_results.csv on the HPC. That file never goes to GitHub.

R2 QC the copilot against DUG Insight

Ruthran · 1 to 2 h

Do	With the app open beside DUG Insight on the HPC, check each well's position, each lead's outline, its trap-type label and its scorecard reasons. List each problem with its lead ID. They become tasks for Edwin or Atif.

R3 Knowledge cards and lead rules

Ruthran · 2 h

Do	Read the 7 cards in knowledge/ and the lead rules in engine/leads.py, with Edwin explaining them. Correct anything wrong. Add a card for any trap or pitfall the copilot gets wrong. Edit on github.com: open the file, click the pencil, start a pull request.

R4 Reference answers and blind scoring

Ruthran + Vincent · 3 h

Do	Write a short reference answer for each of the 30 questions. After the eval run, score each answer blind: grounded 0 to 3, correct 0 to 3, alternatives 0 to 2, clear 0 to 2. Both files stay on the HPC.

R5 Check the story and prepare Q&A

Ruthran · 1 h

Do	Check every geological claim in the slides. Prepare answers to "How did you calibrate?" and "What if the trap is something you didn't model?"

Tasks: Vincent

V4 Email the organisers today

Vincent · 15 min

Ask	May the slides and the public repo show screenshots of the challenge data? Is the demo on our laptop over RustDesk or on venue machines? Is the Gemma endpoint guaranteed during judging? What is the typical cost of an offshore Malaysian exploration well, for the business case?

V1 Deck skeleton and timed script

Vincent · 3 h

Must include	The handbook requires a team introduction, the challenge addressed, a solution overview, demo screenshots and the repository link. The talk is 10 minutes including the demo, then 5 minutes of Q&A.
Timing	0:00 title, 0:45 the problem, 1:45 the approach, 2:45 live demo, 5:15 blind test or look-alikes, 7:00 proof and impact, 8:15 why it's new, 9:15 close with the repo link.
Screenshots	Use the synthetic demo file until the organisers allow real-data screenshots (V4).

V2 Business case

Vincent + Ruthran · 3 h

Do	Show time saved per block and the risked value of the top lead against one avoided dry hole, with EMV = GCoS × value − (1 − GCoS) × dry-hole cost. Every input is sourced (mentor, organiser or a named public source) or labelled as an assumption. Measure the screening time in a rehearsal instead of guessing it.

V3 Draft 30 questions

Vincent · 1.5 h

Do	Use the tiers in eval/questions_template.csv: trap identification, calibration, look-alikes, risks, follow-ups, plain language, and two questions with no evidence behind them.

V5 · V6 Rehearse and submit

Vincent + everyone · 2 h

Do	Three timed rehearsals, the last on the real remote desktop, plus a Q&A drill with Ruthran and Atif. On day 3, submit the slides and the repo link once the organisers share the link.

Each machine

Person	Machine	How the agent gets the rules	Daily start
Shane	Mac, Claude Code	CLAUDE.md imports AGENTS.md. Gates 1 and 2 run here.	git pull, then the HPC terminal line in docs/team-setup.md
Atif	Windows, Codex	Codex reads AGENTS.md at the repo root on its own.	git pull. Tell Codex: "Follow AGENTS.md. Task <ID>. Branch atif/<id>-<name>. Never push to main."
Edwin	Laptop, his agent	Codex, Cursor and Copilot read AGENTS.md. For any other agent, paste it in.	git pull, then the fake SEG-Y loop
Ruthran	HPC desktop, GitHub website	No agent	DUG Insight session Challenge1_Challenge2_InputData
Vincent	Slides tool, GitHub website	No agent	The deck and the synthetic demo file for screenshots

Tests on laptops, before every pull request

On Windows PowerShell, set variables once per terminal, for example $env:MOCK="1", instead of the MOCK=1 prefix.

#	Command	Must show	Who
1	python -m unittest discover -s tests -p "test_*.py"	All pass	Shane, Atif, Edwin
2	node --test tests/test_chat.mjs	3 pass	Shane, Atif
3	python scripts/check_ui.py	All checks passed	Anyone who touched frontend/
4	MOCK=1 python -m engine.precompute MOCK=1 uvicorn backend.app:app --port 8800	App loads, browser checklist passes	Shane, Atif, Edwin
5	MOCK=1 python eval/run_eval.py --questions eval/questions_mock.csv --tag <branch>	10 answers, no tracebacks	Anyone who touched backend/
6	Fake SEG-Y loop (AGENTS.md, Edwin's section)	All checks passed, server answers	Anyone who touched engine/ or config.py
7	MOCK=1 python frontend/build_standalone.py	File written, opens offline	Shane

Browser checklist, about 3 minutes

- The welcome screen loads and opens the workspace. Clicking a lead updates the map, the scorecard and the section.
- "What is the geological opportunity, and why?" streams an answer with evidence tags. Clicking a tag opens its dialog and pins the map.
- "Rank the leads" with a lead selected gives all leads. "Is there gas at inline 5000, crossline 2050?" answers "Location unavailable".
- Executive mode, the blind well test, the committee memo and the light theme open. The console shows no red errors.

Tests on the HPC, after every merge (Shane)

When	Command	Must show
engine/ changed	python scripts/check_real_data.py	No FAIL lines
engine/ changed	python -m engine.precompute --t0 <ms> --t1 <ms>, then python -m engine.render_instrument	Store written
Always	uvicorn backend.app:app --host 127.0.0.1 --port 8800, then curl -s http://127.0.0.1:8800/api/health	mock false, llm_mode gemma, llm_reachable true
Always	Browser checklist in the HPC's Firefox	Footers say "Gemma 4". Ruthran agrees that wells and leads match DUG Insight.

Never

- Commit seismic, well or store files, data/well_results.csv, eval/questions.csv, screenshots of real maps, HPC logins, the HPC manual, the RustDesk key or tokens. The repo is public.
- Copy challenge data or anything built from it off the HPC, including answers, eval sheets and screenshots, until the organisers allow screenshots.
- Push to main or merge your own pull request. Coding agents work on branches only.
- Invent numbers in answers, on screen or in slides. Every number comes from the store, the user's inputs or a named source.
- Edit code on the HPC without pushing it straight away, or start a second server there.
- Start a task without a gate 1 go, or change a shared contract without gate 3.